Adaptive Layer-wise Thresholding and Learnable Cosine Distance Metrics for Robust Graph Neural Networks

Tawkir Aziz Rahman[1] Dipanta Kumar Roy Nobo[1]

[1]Bangladesh University of Engineering and Technology
Dhaka, Bangladesh

https://github.com/TAR2003/RUNG_ML_Project

2026

Outline

1. Introduction & Motivation
2. Methodology
3. Experimental Evaluation
4. Robustness Curves
5. Analysis & Discussion
6. Limitations & Future Work
7. Conclusion

Graph Neural Networks & Adversarial Vulnerability

GNN Message Passing

- Each node aggregates features from its neighborhood over K layers
- Fundamental architecture for citation, social, and molecular graphs
- **Critical vulnerability:** susceptible to adversarial structural attacks

Threat Model (Adaptive PGD Attack)

- Adversary has **full knowledge** of model and parameters
- Budget $\delta \in [0, 1]$: fraction of original edges that may be added/deleted
- Attack gradient flows through entire GNN architecture
- **Feature vectors are not modified** (structural-only attack)

Formal Problem

Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, n nodes, m edges

$\mathbf{X} \in \mathbb{R}^{n \times d_0}$: node feature matrix

$\mathbf{A} \in \{0, 1\}^{n \times n}$: adjacency matrix

Attacker yields perturbed $\tilde{\mathbf{A}}$ with at most $\delta \cdot m$ edge changes

Goal: minimize GNN classification accuracy

RUNG: State-of-the-Art Robust GNN (NeurIPS 2024)

RUNG's Core Innovation (RUGE)

- Applies **Minimax Concave Penalty (MCP)** to eliminate estimation bias in ℓ_1-regularized graph smoothing
- Prunes edges where feature differences exceed learnable threshold γ
- Strong robustness under large adaptive budgets
- Time complexity: $\mathcal{O}(K(m+n)d)$

Two Critical Limitations of RUNG

1. **Static threshold γ:** fixed at initialization, shared across all K layers — becomes misaligned as features smooth in deeper layers
2. **Euclidean distance:** sensitive to magnitude; uses `.detach()`, breaking the gradient chain — prevents adaptive geometry learning

RUNG's Euclidean Metric

$$y_{ij}^{(k)} = \left\| \frac{\mathbf{f}_i^{(k)}}{\sqrt{d_i}} - \frac{\mathbf{f}_j^{(k)}}{\sqrt{d_j}} \right\|_2^2$$

- Clusters near identical distances in high dimensions (curse of dimensionality)
- Disconnected from computation graph via `.detach()`
- Fails on heterophilic graphs where legitimate edges connect dissimilar nodes

Our Contributions: Two Complementary Enhancements

Both methods operate within the RUGE framework, modifying *only* the edge-weight computation $W_{ij}^{(k)}$ at each layer.

Method 1: Adaptive Percentile-based Thresholding

$$\gamma^{(k)} = \text{quantile}\Big(\{y_{ij}^{(k)} : (i,j) \in \mathcal{E}\},\ q\Big)$$

- Default $q = 0.75$ (top 25% highest-distance edges pruned)
- **No learnable parameters added**
- Automatically tracks feature-scale changes across layers
- As attack budget grows, distribution tail shifts up $\Rightarrow \gamma^{(k)}$ increases automatically
- Computational cost: $\mathcal{O}(m \log m)$ per layer — dominated by $\mathcal{O}(md)$

Method 2: Learnable Cosine-Distance Modules

$$y_{ij}^{(k)} = 1 - \frac{\mathbf{f}_i^{(k)} \cdot \mathbf{f}_j^{(k)}}{\|\mathbf{f}_i^{(k)}\|_2 \cdot \|\mathbf{f}_j^{(k)}\|_2}$$

- **Invariant to scaling:** attacker cannot reduce distance by matching magnitudes alone
- Removes `.detach()` $\Rightarrow$ gradient flows through distance computation
- Network learns projections where legitimate edges are *angularly close*, adversarial edges *angularly distant*
- Combined with percentile thresholding ($q = 0.75$)

Gradient Flow in Cosine Distance

Gradient of Cosine Distance

$$\nabla_{\mathbf{f}_i} y_{ij}^{(k)} = \frac{\mathbf{f}_j^{(k)} / \|\mathbf{f}_j^{(k)}\|_2}{\|\mathbf{f}_i^{(k)}\|_2} - \frac{\mathbf{f}_i^{(k)} \left(\mathbf{f}_i^{(k)} \cdot \mathbf{f}_j^{(k)}\right)}{\|\mathbf{f}_i^{(k)}\|_2^3 \|\mathbf{f}_j^{(k)}\|_2}$$

Geometry Learning

- Network learns an **optimal projection subspace** for distinguishing valid from spurious connections
- Learned geometry is optimized specifically for the given graph
- Encodes the graph's class homophily structure into feature space
- This learning signal is **fundamentally absent** in RUNG's hand-crafted, non-adaptive Euclidean distance

Why Cosine Helps Heterophilic Graphs

- RUNG assumes feature dissimilarity $\Rightarrow$ adversarial edge
- On heterophilic graphs (e.g., Chameleon: 77% cross-class edges), legitimate connections naturally have large Euclidean distances
- **Cosine distance** captures directional dissimilarity, not magnitude difference
- Distinguishes:
  - Genuine heterophilic edges: different directions but legitimately connected
  - Adversarial edges: larger cosine distance after learning

Complexity Analysis

Theorem (Complexity Preservation)

Both the percentile-gamma and learnable cosine-distance methods preserve the $\mathcal{O}(K(m+n)d)$ time complexity of the original RUNG framework.

Per-Layer Cost Breakdown

1. **Edge-weight computation:** $\mathcal{O}(md)$ for all m edges (Euclidean or cosine)
2. **Quantile computation** (Method 1 only): $\mathcal{O}(m \log m)$ — for any $d \geq 2$, dominated by $\mathcal{O}(md)$
3. **Diagonal inversion & aggregation:** $\mathcal{O}(n) + \mathcal{O}(md)$
4. **Feature regularization:** $\mathcal{O}(nd)$

Total across K layers: $\mathcal{O}(K(m+n)d)$ ✓

Practical Speedup Note

- Cosine distance on **sparse graphs** ($m \ll n^2$) is practically faster than RUNG's pairwise Euclidean computation
- Realized through sparse matrix operations on modern GPUs (CUDA batched sparse dot products)
- No significant computational overhead introduced

Experimental Setup

Benchmark Datasets

Dataset	Nodes	Edges	Cls.	Homoph.
Homophilic				
Cora	2,708	5,429	7	0.81
Citeseer	3,327	4,732	6	0.74
Pubmed	19,717	44,338	3	0.80
Heterophilic				
Chameleon	2,277	36,101	5	0.23
Wisconsin	251	499	5	0.21
Cornell	183	295	5	0.30

Experimental Configuration

- PGD attack from original RUNG codebase
- Standard train/val/test splits
- **10 independent runs**, different random seeds; mean $\pm$ std reported
- Budgets $\delta \in \{0.1, 0.2, \ldots, 1.0\}$

Models Compared

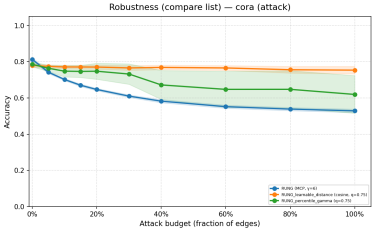
- **RUNG (baseline)** — fixed $\gamma = 6$
- **RUNG-PercentileGamma** ($q = 0.75$) — Method 1
- **RUNG-CosineDistance** (cosine, $q = 0.75$) — Method 2

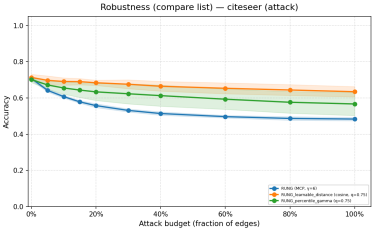
Quantitative Results: Clean & Maximum-Budget Accuracy

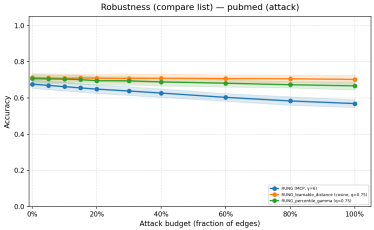
Classification Accuracy under Clean and Fully Adversarial ($\delta = 1.0$) Conditions — Mean $\pm$ Std over 10 Seeds

		Clean Accuracy		100% Budget Accuracy	
Dataset	**Model**	Mean	Std	Mean	Std
Cora	RUNG (baseline)	81.33%	$\pm$0.87	52.92%	$\pm$0.97
	RUNG-PercentileGamma	78.59%	$\pm$1.38	61.97%	$\pm$10.41
	RUNG-CosineDistance	78.05%	$\pm$0.96	**75.38%**	$\pm$2.10
Citeseer	RUNG (baseline)	70.79%	$\pm$0.91	48.38%	$\pm$0.94
	RUNG-PercentileGamma	70.19%	$\pm$1.63	56.71%	$\pm$6.37
	RUNG-CosineDistance	71.34%	$\pm$1.74	**63.47%**	$\pm$2.83
Pubmed	RUNG (baseline)	67.64%	$\pm$2.29	56.93%	$\pm$2.16
	RUNG-PercentileGamma	70.66%	$\pm$2.36	66.72%	$\pm$2.27
	RUNG-CosineDistance	71.27%	$\pm$2.24	**70.26%**	$\pm$2.44
Chameleon	RUNG (baseline)	41.53%	$\pm$1.05	14.52%	$\pm$2.53
	RUNG-PercentileGamma	39.44%	$\pm$0.72	32.29%	$\pm$5.46
	RUNG-CosineDistance	**44.27%**	$\pm$1.18	**40.48%**	$\pm$4.44
Wisconsin	RUNG (baseline)	53.13%	$\pm$7.50	34.23%	$\pm$6.54
	RUNG-PercentileGamma	59.00%	$\pm$3.78	49.95%	$\pm$3.76
	RUNG-CosineDistance	**59.20%**	$\pm$4.47	**61.19%**	$\pm$3.37
Cornell	RUNG (baseline)	45.17%	$\pm$5.77	30.34%	$\pm$7.21
	RUNG-PercentileGamma	44.90%	$\pm$0.43	41.63%	$\pm$4.09
	RUNG-CosineDistance	**49.93%**	$\pm$3.98	**47.62%**	$\pm$2.43

Robustness Degradation Curves — Homophilic Graphs







Cora (Homophily: 0.81)

RUNG: 81% → 53%
RUNG-PercentileGamma: maintains 62%
RUNG-CosineDistance: **75%** (93% of clean acc.)

Citeseer (Homophily: 0.74)

RUNG: drops 22.4 pts
RUNG-CosineDistance: degrades only 7.9 pts

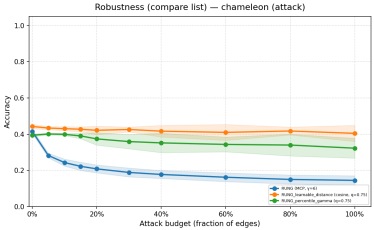
Pubmed (Homophily: 0.80)

All methods more stable here
RUNG-CosineDistance: maintains ≈70% accuracy across all budgets

Observation

On homophilic graphs, all three methods degrade monotonically with increasing budget, but proposed methods maintain substantially higher accuracy at high perturbation budgets.

Robustness Degradation Curves — Heterophilic Graphs

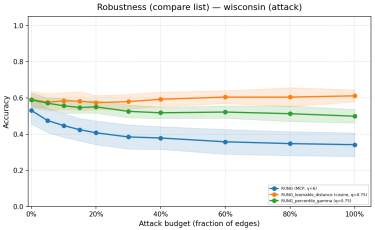


Chameleon (Homophily: 0.23)

RUNG: 41.5% → 14.5% (catastrophic)

RUNG-CosineDistance: **44.3% → 40.5%**

+25.9 pp absolute improvement

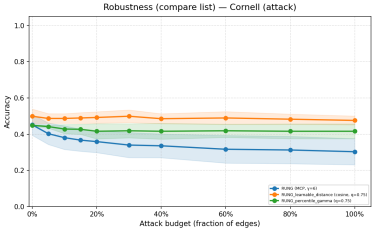


Wisconsin (Homophily: 0.21)

RUNG: drops 19 pts

RUNG-CosineDistance: **59% → 61%**

Accuracy *rises* under attack!



Cornell (Homophily: 0.30)

RUNG: drops 14.8 pts

RUNG-CosineDistance: degrades only **2.3 pts**

+12.5 pp stabilization

Key Finding

RUNG's curve steepens dramatically in the 20%–50% budget range on heterophilic graphs. RUNG-CosineDistance exhibits **flat degradation**, indicating fundamental architectural alignment with heterophilic structure.

Why RUNG Fails on Heterophilic Graphs

RUNG's Flawed Assumption

- RUNG assumes: **feature dissimilarity** $\Rightarrow$ adversarial edge
- Valid on homophilic graphs (legitimate edges connect similar nodes)
- **Violated on heterophilic graphs** by genuine network structure

Chameleon Example (77% cross-class edges)

- RUNG's MCP filter assigns zero weight to large-distance edges
- Legitimate cross-class edges naturally have large feature distances
- RUNG prunes valid edges, weakening graph structure utilization
- Attacker's task becomes *easier*: large-distance fake edges are harmless when RUNG is already deleting most high-distance edges

How Cosine Distance Resolves This

- Measures **orientation**, not magnitude
- Directional alignment is invariant to feature magnitude
- Can distinguish:
  - **Genuine heterophilic edges:** different directions, legitimately connected (small cosine dist. after learning)
  - **Adversarial edges:** larger cosine dist. after network learns to distinguish

Wisconsin Phenomenon

RUNG-CosineDistance accuracy *increases* under moderate attack (0%–60% budget), suggesting adversarial edges may initially provide additional structural information that the cosine metric effectively filters.

Layer-wise Adaptive Threshold Analysis (Cora)

Euclidean Distance Thresholds (Method 1)

Layer k	$\gamma^{(k)}$
0	3.6571
1	3.2985
2	3.1846
3	3.1407
4	3.1705
5	3.1592
6	3.1544
7	3.1394
8	3.1445
9	3.1406

Threshold **decays** from initial to deeper layers as feature smoothing occurs.

Cosine Distance Statistics (Method 2)

Layer	$\gamma^{(k)}$	**mean**	**std**	**max**
0	0.3755	0.7520	0.3903	1.7692
1	0.2494	0.7485	0.3962	1.7407
2	0.2321	0.7499	0.3999	1.7128
3	0.2234	0.7524	0.4012	1.6956
4	0.2209	0.7527	0.4020	1.6956
5	0.2115	0.7534	0.4022	1.6956
6	0.2132	0.7536	0.4025	1.6956
7	0.2065	0.7542	0.4027	1.6956
8	0.2124	0.7544	0.4029	1.6956
9	0.2030	0.7547	0.4030	1.6956

Both $\gamma^{(k)}$ and mean cosine distance decrease with depth, confirming that features become progressively more aligned across layers.

Threat Model Constraints: The Feature-Aware Attack

Limitation: Structural-Only Threat Model

Current evaluation assumes the attacker can only modify graph topology. A **fully adaptive adversary with feature perturbations** exposes a fundamental vulnerability.

The Feature-Spoofing Attack Mechanism

1. Attacker observes learned feature representation of target node v_{target}
2. Sets adversarial node feature $\mathbf{x}_{\text{adv}} \approx \mathbf{x}_{\text{target}}$
3. Adds edge connecting them
4. Both Euclidean and cosine distance $\to 0$
5. QN-IRLS assigns **maximum weight** to the adversarial edge

If δ_f large enough: $W^{(k)}_{\text{adv,target}} = \text{MCP}(y^{(k)} \approx 0) \to 1$

Why Practical Constraints Limit This Attack

- **Citation networks:** Cannot change paper title/abstract without detectable inconsistencies — $\approx 0.3\%$ feature budget
- **Social networks:** Fake account creation dates, follower counts, posting history trigger detection systems
- **Biological networks:** Cannot synthesize proteins with arbitrary sequences

Small $\delta_f < 0.1$: distance remains large, defense holds.
Large $\delta_f > 0.3$: defense **collapses entirely**.

Future Work Directions

1. Feature-Aware Adversarial Attacks

Implement joint edge-and-feature attack:

$$\mathcal{A}_{\text{joint}} = \{(\tilde{A}, \tilde{X}) : \Delta_{\text{edge}} + \Delta_{\text{feature}} \leq B\}$$

Craft specifically to minimize intended edge's cosine distance while keeping nodes dissimilar in feature content.

2. Theoretical Robustness Analysis

Analyze whether MCP-based estimators applied to cosine distances maintain the **unbiasedness property** from the original RUNG analysis. Extend Hou et al. [NeurIPS 2024] to non-Euclidean metrics.

3. Scalability Improvements

- Distributed quantile computation and sparse cosine distance kernels for multi-GPU inference
- Benchmark on large-scale graphs (Ogbn-Arxiv: 1.63M nodes, 40.9M edges)
- Current evaluation limited to moderate scale (Wisconsin: 251 nodes; Pubmed: 19,717 nodes)

4. Learnable Distance Metrics Beyond Cosine

Extend to bilinear and projection-based metrics with trainable parameters for further geometry adaptation to specific graph structures.

Conclusion

Summary of Contributions

Two complementary enhancements to RUNG (NeurIPS 2024 state-of-the-art):

- **Method 1 — Adaptive Percentile-based Thresholding:** Layer-wise pruning thresholds from empirical edge-distance quantiles; no learnable parameters; automatic scale tracking
- **Method 2 — Learnable Cosine-Distance Modules:** Angle-based distance metric; gradient-driven feature learning; optimal subspace discovery for legitimate vs. adversarial edge discrimination

Key Findings

- **Homophilic graphs:** +22.5 pp (Cora), +15.1 pp (Citeseer), +13.3 pp (Pubmed) at maximum budget
- **Heterophilic graphs:** RUNG's 27-pt degradation on Chameleon reduced to 3.8 pts; Wisconsin accuracy *rises* under attack
- Both methods outperform RUNG on **all six datasets**

Open Problems

- No formal theoretical guarantees for adaptive thresholding or cosine geometry
- Feature-aware adversary can bypass distance-based defenses
- Large-scale evaluation not yet conducted
- These gaps represent important future research directions

Heterophilic-robust graph learning is achievable through adaptive per-layer thresholding combined with learnable angle-based distance metrics.

References

1. Hou et al. *Robust Graph Neural Networks via Unbiased Aggregation*. NeurIPS 2024.
2. Li et al. *Towards Robust Graph Neural Networks*. IEEE ICDM 2020.
3. Luan et al. *Towards Deeper Graph Neural Networks with Differentially Private Learning*. ICML 2022.
4. Sun et al. *Adversarial Attacks on Graphs: Deep Insights and Best Practices*. IJCAI 2021.
5. Tam et al. *Defending Against Adversarial Attacks by Reconstruction*. NeurIPS 2021.
6. Xu et al. *Topology Attack and Defense for Graph Neural Networks: An Optimization Perspective*. IJCAI 2019.
7. Yan et al. *Heterophilic Graph Neural Network*. NeurIPS 2021.
8. Zhang. *Nearly Unbiased Variable Selection Under Minimax Concave Penalty*. Annals of Statistics 2010.
9. Zhu et al. *Beyond Homophily in Graph Neural Networks*. NeurIPS 2020.
10. Zhu et al. *GNNGuard: Defending GNNs against Adversarial Attacks*. NeurIPS 2020.
11. Zugner et al. *Adversarial Attacks on Neural Networks for Graph Data*. KDD 2018.
12. Zugner & Günnemann. *Certifiable Robustness and Robust Training of GCNs*. KDD 2019.